



UNIVERSITETI I PRISHTINËS “HASAN PRISHTINA”

Fakulteti i Inxhinierisë Elektrike dhe Kompjuterike

Inxhinieri Kompjuterike dhe Softuerike – Master

Smart Health Monitoring

Dokumentimi i Projektit 2 – Ndërtimi i një sistemi IoT

Lënda: Internet of Things

Profesori: Besmir Sejdiu

Grupi 2

Studentët:

Rinor Ukshini

Rina Bunjaku

Arton Gërguri

Prishtinë, 2026

Përmbajtja

1. Hyrje	3
2. Infrastruktura e Projektit	4
3. Integrimi me Apache Kafka.....	5
4. Procesimi me Apache Spark Streaming.....	6
5. Ruajtja e të Dhënave në Apache Cassandra	7
6. Ndërfaqja e Vizualizimit	7
7. Komponentët e Avancuar.....	11
7.1 Inteligjenca Artificiale	11
7.2 Sistemi i Alarmeve.....	11
7.3 Analiza e Performancës.....	11
8. Sfidat dhe Mësimet e Nxjerra	11
9. Përfundime dhe Rekomandime	12
10. Puna e Ardhshme	12

Hyrje

Ky raport dokumenton projektin Smart Health Monitoring, një sistem IoT i plotë i zhvilluar në kuadër të lëndës Internet of Things. Qëllimi i projektit është të demonstrojë një infrastrukturë të plotë në të cilën të dhënat e sensorëve mbledhen, transmetohen, përpunohen në kohë reale dhe ruhen në mënyrë të organizuar, duke u vizualizuar përmes ndërfaqes vue-client (Vue 3 SPA), e servuar nga backend-i Vue.Api.

Domeni i zgjedhur është monitorimi i shëndetit në një mjedis spitalor. Sistemi monitoron vazhdimisht dhjetë pacientë të shtrirë në dhomat 101–110. Në mungesë të sensorëve fizikë, është ndërtuar një simulator që gjeneron lexime realiste të parametrave vitalë çdo rreth dy sekonda.

Objektivat kryesore të projektit:

- Mbledhja e të dhënave të parametrave vitalë nga sensorë të simuluar.
- Transmetimi i të dhënave në kohë reale përmes Apache Kafka.
- Përpunimi i rrjedhës së të dhënave me Apache Spark Structured Streaming.
- Ruajtja e të dhënave të papërpunuara dhe të agreguara në Apache Cassandra.
- Vizualizimi i të dhënave përmes ndërfaqes vue-client (Vue 3, Vite, Pinia, ApexCharts), e integruar me Vue.Api përmes REST API dhe SignalR.
- Implementimi i komponentëve të avancuar: inteligjenca artificiale, sistemi i alarmeve dhe analiza e performancës.

Parametrat vitalë që maten për secilin pacient janë: pulsi (BPM), saturimi i oksigjenit (SpO_2), temperatura trupore, tensioni arterial (sistolik dhe diastolik) dhe frekuenca e frymëmarrjes.

Infrastruktura e Projektit

Sistemi është ndërtuar si një arkitekturë modulare ku secili komponent ka një përgjegjësi të qartë dhe komunikon me të tjerët përmes ndërfaqeve të mirëpërcaktuara. Të dhënat rrjedhin në një drejtim të vetëm: nga simulatori i sensorëve, te shtresa e transmetimit (Kafka), te përpunimi në kohë reale (Spark Streaming), te baza e të dhënave (Cassandra) dhe në fund te ndërfaqja vue-client.

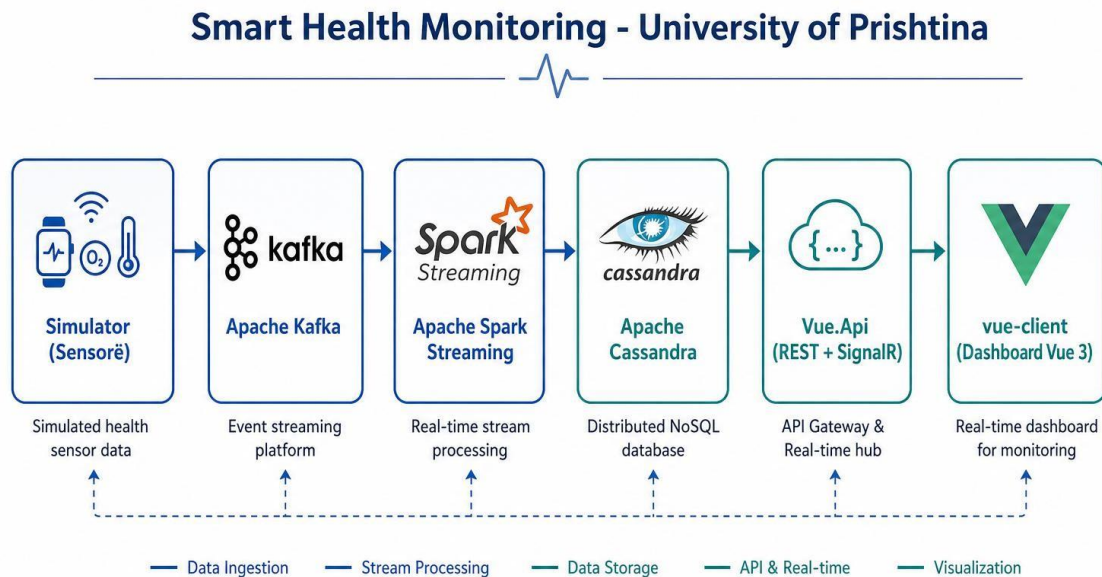


Figura 1: Diagrami i arkitekturës së sistemit Smart Health Monitoring

Rrjedha e përgjithshme e të dhënave:

Simulator.App gjeneron leximet vitale për 10 pacientë (dhomat 101–110) çdo ~2 sekonda dhe i dërgon te Apache Kafka si mesazhe JSON. Streaming.App i konsumon ato me Apache Spark Structured Streaming (ose në mënyrën alternative “direct” me Confluent.Kafka), i filtron, i agregon në dritare kohore, vlerëson rrezikun, gjeneron alarme dhe i ruan në Apache Cassandra. Njëkohësisht, Streaming.App njofton ndërfaqen vue-client në kohë reale përmes SignalR (hub-i /healthHub në Vue.Api), duke dërguar vitale, alarme, vlerësim rreziku dhe metrika. Vue.Api lexon të dhënat historike nga Cassandra përmes REST API-së (/api/overview, /api/analytics, /api/patients/{id}, etj.), ndërsa vue-client i shfaq ato në shtatë faqe funksionale.

Modulet kryesore dhe roli i tyre:

Moduli	Teknologjia	Roli
Simulator.App	.NET 8 Worker	Gjeneron parametra vitalë realistë dhe i publikon në Kafka
Streaming.App	Apache Spark 3.2 + .NET	Konsumon Kafka me Spark Structured Streaming; filtron, agregon, alarmon, vlerëson rrezikun dhe ruan në Cassandra
Vue.Api	ASP.NET Core 8	Backend për vue-client: REST API, hub SignalR (/healthHub) dhe lexim historik nga Cassandra
vue-client	Vue 3, Vite, Pinia, ApexCharts	Ndërfaqja SPA e vizualizimit (vue-client) — monitorim live dhe analitika
AI.Training	ML.NET	Trajnimi offline i modelit parashikues të infarktit

I gjithë infrastruktura bazë orkestrohet përmes Docker Compose (Zookeeper :2181, Kafka :9092, Cassandra :9042). Pas ngritjes së kontejnerëve, skema e Cassandra inicializohet me cqllsh. Më pas nisen Vue.Api (porti 5099), Streaming.App dhe Simulator.App sipas rendit: infrastruktura → API → përpunues → burimi i të dhënave.

Integrimi me Apache Kafka

Apache Kafka shërben si shtresa qendrore e transmetimit të mesazheve. Infrastruktura ngritet me docker compose up -d, e cila starton Zookeeper, Kafka, Cassandra dhe krijon automatikisht topic-un health-vitals (3 partitions, replication factor 1). Të dhënat e gjeneruara nga simulatori dërgohen te ky topic me çelës mesazhi patientId.

Komponentët në lidhje me Kafka:

Komponenti	Roli
Producer	Simulator.App (.NET 8 Worker) publikon leximet vitale në topic-un health-vitals çdo ~2 s
Consumer	Streaming.App lexon rrjedhën për përpunim me Spark Structured Streaming ose në mënyrën direct
Topic	health-vitals — 3 partitions, replication factor 1, message key: patientId, format: JSON

Çdo mesazh serializohet në format JSON dhe përmban identifikuesin e pacientit, dhomën, moshën dhe vlerat e parametrave vitalë. Rreth 5–8% e leximeve të simulatorit janë të qëllimshme anormale, për të demonstruar alarmet dhe vlerësimin e rrezikut. Një shembull i mesazhit:

```
{
  "patientId": "P001", "patientName": "Arben
  Krasniqi", "roomNumber": "101",
  "heartRate": 88,
  "spo2": 97,
  "temperature": 36.8,
  "systolicBp": 128,
  "diastolicBp": 82,
```

```
"respiratoryRate": 16, "recordedAt": "2026-06-11T10:30:00Z"
}
```

Pse Apache Kafka:

- Decoupling: simulatori dhe përpunuesi nuk varen drejtpërdrejt nga njëri-tjetri.
- Buffering: të dhënat ruhen përkohësisht edhe nëse përpunimi ndalon për pak.
- Shkallzueshmëria: ndarjet e topic-ut mundësojnë përpunim paralel dhe rritje të ngarkesës.
- Përpunim në kohë reale: rrjedhë e vazhdueshme mesazhesh tipike për sistemet IoT.
- Konfigurimi i producer-it (Simulator.App): BootstrapServers, ClientId smart-health-simulator, Acks=Leader. Consumer-i (Streaming.App) përdor GroupId smart-health-direct-streaming në mënyrën direct.

Hapat për konfigurimin dhe përdorimin e Kafka

- Hapi 1 – Ngritja e infrastrukturës me Docker Compose. Në projekt ekzekutohet komanda docker compose up -d. Kjo starton kontejnerët smart-health-zookeeper (porti 2181), smart-health-kafka (porti 9092), smart-health-cassandra (porti 9042) dhe shërbimin kafka-init, i cili krijon automatikisht topic-un health-vitals me 3 partitions dhe replication factor 1.
- Hapi 2 – Verifikimi i topic-ut. Pas ngritjes së Kafka, topic-u health-vitals duhet të ekzistojë. Verifikimi bëhet me komandën: kafka-topics --bootstrap-server localhost:9092 --list. Duhet të shfaqet health-vitals midis topic-ve të tjera.
- Hapi 3 – Konfigurimi i aplikacioneve .NET. Të dy modulet Simulator.App dhe Streaming.App lexojnë parametrat e Kafka nga skedari appsettings.json, seksioni Kafka: BootstrapServers (adresa e broker-it, p.sh. localhost:9092 ose IP e serverit) dhe Topic (health-vitals).
- Hapi 4 – Nisja e producer-it (Simulator.App). Ekzekutohet me dotnet run --project Simulator.App. Worker-i krijon një Kafka Producer (Confluent.Kafka) me ClientId smart-health-simulator dhe Acks=Leader, pastaj publikon mesazhe JSON çdo ~2 sekonda për secilin nga 10 pacientët, duke përdorur patientId si çelës mesazhi.
- Hapi 5 – Nisja e consumer-it (Streaming.App). Pas producer-it, nisët Streaming.App që subscribohet në topic-un health-vitals dhe fillon përpunimin e rrjedhës (Spark Structured Streaming ose mënyra direct). Consumer-i përdor GroupId smart-health-direct-streaming dhe AutoOffsetReset=Latest në mënyrën direct.
- Hapi 6 – Monitorimi i rrjedhës. Mesazhet mund të verifikohen me kafka-console-consumer --bootstrap-server localhost:9092 --topic health-vitals --from-beginning. Çdo rresht përfaqëson një lexim vital JSON me fushat patientId, roomNumber, heartRate, spo2, temperature, systolicBp, diastolicBp, respiratoryRate dhe recordedAt.
- Përdorimi praktik: rendi i nisjes së sistemit është: (1) docker compose up -d, (2) inicializimi i Cassandra me cqlsh, (3) Vue.Api, (4) Streaming.App, (5) Simulator.App. Kafka vepron si buffer ndërmjet burimit të të dhënave dhe përpunuesit, duke mundësuar rifillim të consumer-it pa humbje të mesazheve të ruajtura në topic.

Procesimi me Apache Spark Streaming

Zemra e përpunimit është Streaming.App, e ndërtuar mbi Apache Spark Structured Streaming (Microsoft.Spark 2.1.1, SparkSession “Smart Health Monitoring Streaming”). Aplikacioni lexon rrjedhën nga topic-u health-vitals dhe e përpunon në kohë reale para se ta ruajë në Cassandra dhe ta publikojë te vue-client përmes SignalR.

Hapat kryesorë të përpunimit:

1. Leximi i rrjedhës nga Kafka përmes ReadStream të Spark.
2. Filtrimi dhe validimi: hidhen poshtë leximet e pavlefshme ose klinikisht të pamundshme (p.sh. puls jashtë intervalit 0–260 ose SpO₂ jashtë 0–100).
3. Agregimi në dritare rrëshqitëse prej 5 minutash me hap (slide) prej 1 minute, duke përdorur watermark për tolerimin e të dhënave që vijnë jashtë rendit.
4. Ruajtja e leximeve të papërpunuara dhe e mesatareve të dritareve në Cassandra.
5. Njoftimi i ndërfaqes në kohë reale përmes SignalR, si dhe vlerësimi i rrezikut dhe gjenerimi i alarmeve.
6. Për çdo lexim valid, ProcessReadingAsync: (1) ruan vitalin dhe profilin e pacientit në Cassandra; (2) publikon vitale te vue-client; (3) llogarit agregimin e dritares; (4) vlerëson rrezikun me RiskScoringEngine; (5) vlerëson alarmet me AlertRules dhe i publikon live.

Dritarja rrëshqitëse (5 minuta, slide 1 minutë, watermark 10 minuta) llogarit mesataret e parametrave vitalë (puls, SpO₂, temperaturë, tension, frymëmarrje) për çdo pacient dhe dhomë. Rezultatet ruhen në tabelën vitals_window_agg dhe shfaqen në faqen Analitika të vue-client.

Spark ekzekutohet në mënyrën kryesore përmes spark-submit (JVM bridge i Microsoft.Spark). Komanda e nisjes: spark-submit --class org.apache.spark.deploy.dotnet.DotnetRunner --master local microsoft-spark-*.jar dotnet Streaming.App.dll --spark. Për mjedise me burime të kufizuara ekziston edhe mënyra “direct” (dotnet run --project Streaming.App), e cila përdor Confluent.Kafka Consumer pa JVM, por me të njëjtin pipeline logjik: validim → ruajtje → agregim → alarm → SignalR.

Detajet e implementimit të procesimit në kohë reale

Streaming.App implementohet në C# (.NET 8) duke përdorur bibliotekën Microsoft.Spark 2.1.1. Aplikacioni mbështet dy mënyra pune: Spark Structured Streaming (me flag --spark dhe spark-submit) dhe mënyra direct (Confluent.Kafka Consumer pa JVM). Të dyja përdorin të njëjtën logjikë përpunimi përmes funksioneve ProcessReadingAsync dhe StoreWindowBatchAsync.

2.1. Krijimi i SparkSession. Në mënyrën Spark, krijohet SparkSession me emrin Smart Health Monitoring Streaming. Ky session menaxhon dy query paralele të Structured Streaming: një për procesimin për lexim dhe një për agregimin me dritare rrëshqitëse.

2.2. Leximi nga Kafka (ReadStream). Burimi i të dhënave konfigurohet me format kafka, opsionet kafka.bootstrap.servers (nga appsettings), subscribe=health-vitals dhe startingOffsets=latest. Kolona value dekodohet nga bytes në string JSON.

2.3. Deserializimi dhe skema. Mesazhi JSON parse-ohet me FromJson duke përdorur skemën: patientId, patientName, roomNumber, age, heartRate, spo2, temperature, systolicBp, diastolicBp, respiratoryRate, recordedAt (të gjitha me tipet përkatëse Spark SQL).

2.4. Filtrimi dhe validimi. Pas deserializimit, aplikohet Filter që heq leximet e pavlefshme: patientId IS NOT NULL, heartRate > 0 AND heartRate < 260, spo2 > 0 AND spo2 <= 100. Kjo garanton që vetëm lexime klinike të mundshme kalojnë në fazat e mëpashtme.

2.5. Procesimi për-lexim (ForeachBatch). Një WriteStream me ForeachBatch thërret ProcessBatchAsync për çdo batch Spark. Për secilin rresht valid, ProcessReadingAsync kryen pesë veprime: (a) InsertVitalAsync dhe UpsertPatientAsync në Cassandra; (b) PublishVitals te vue-client përmes SignalR; (c) agregim lokal i dritares (në mënyrën direct) ose delegim te query e dritares (Spark); (d) vlerësim rreziku me RiskScoringEngine → InsertRiskAsync; (e) vlerësim alarmesh me AlertRules → InsertAlertAsync dhe PublishAlert.

2.6. Agregimi me dritare rrëshqitëse. Query e dytë aplikon WithWatermark(recordedAt, 10 minutes), pastaj GroupBy(Window(recordedAt, 5 minutes, 1 minute), roomNumber, patientId) dhe Agg(Avg për çdo metrikë + Count). Rezultati shkruhet me OutputMode update dhe ForeachBatch → StoreWindowBatchAsync → InsertWindowAsync në tabelën vitals_window_agg.

2.7. Mënyra direct (pa Spark). Kur aplikacioni nisët pa flag --spark, përdoret ConsumerBuilder<string,string> me Confluent.Kafka. Çdo mesazh deserializohet me System.Text.Json në objekt VitalReading, validohet me IsValid() dhe përpunohet menjëherë me ProcessReadingAsync. WindowAggregator lokal zëvendëson agregimin Spark për demo në mjedise me burime të kufizuara.

2.8. Komanda e nisjes me Spark. spark-submit --class org.apache.spark.deploy.dotnet.DotnetRunner --master local microsoft-spark-3.2.x.jar dotnet Streaming.App.dll --spark. Pas nisjes, aplikacioni lidhet me hub-in SignalR të Vue.Api dhe fillon dy streaming query paralele derisa ndërpritet me AwaitTermination().

2.9. Metrikat e performancës. MetricsCollector regjistron mesazhe të procesuara, madhësinë e batch-it, kohën e përpunimit (ms), dritare të agreguara dhe alarme. Çdo 3 sekonda, një snapshot dërgohet te faqja Shëndeti i sistemit të vue-client përmes PublishMetrics.

Ruajtja e të Dhënave në Apache Cassandra

Të dhënat e përpunuara ruhen në Apache Cassandra, në keyspace-in smart_health (SimpleStrategy, replication_factor: 1). Cassandra u zgjodh për shkak të shkrimeve të shpejta me volum të lartë dhe përshtatshmërisë me të dhënat time-series të një sistemi IoT. Skema inicializohet me: `docker exec -it smart-health-cassandra cqlsh -f /schema/cassandra-schema.cql`

Skema përmban gjashtë tabela:

Tabela	Përmbajtja
patient_vitals	Leximet e papërpunuara të parametrave vitalë
patients	Regjistri dhe profili klinik i pacientëve
alerts_log	Historiku i alarmeve (KRITIK / PARALAJMËRIM / INFO)
vitals_window_agg	Mesataret e ditareve 5-minutëshe të llogaritura nga Spark
ai_risk_scores	Rezultati i vlerësimit të rrezikut në kohë reale
ml_predictions	Parashikimet e modelit ML.NET për rrezikun e infarktut

Modelimi i tabelave është bërë sipas pyetjeve që ekzekuton vue-client përmes REST API-së së Vue.Api (parimi query-first i Cassandra). Shembull: patient_vitals përdor PRIMARY KEY ((room_number, vital_day), recorded_at, patient_id) me CLUSTERING ORDER BY (recorded_at DESC), që lejon lexim të shpejtë të historikut sipas dhomës dhe ditës. Streaming.App kryen shkrimet; Vue.Api kryen leximet për dashboard-in.

Udhëzimet për konfigurimin e Cassandra dhe integrimin me Spark Streaming

Hapi 1 – Ngritja e Cassandra me Docker. Shërbimi cassandra në docker-compose.yml përdor imazhin cassandra:4.1, cluster SmartHealthCluster, datacenter datacenter1, porti 9042. Volumi cassandra-data ruan të dhënat ndërmjet rindezjeve; skedari cassandra-schema.cql mount-ohet në /schema/cassandra-schema.cql.

Hapi 2 – Inicializimi i skemës. Pas startimit të kontejnerit (30–60 s), ekzekutohet: `docker exec -it smart-health-cassandra cqlsh -f /schema/cassandra-schema.cql`. Kjo krijon keyspace-in smart_health me SimpleStrategy dhe replication_factor 1, pastaj gjashtë tabela.

Hapi 3 – Konfigurimi në Streaming.App. Seksioni Cassandra i appsettings.json përmban ContactPoint, Port (9042) dhe Keyspace (smart_health). Klasa CassandraHealthRepository lidhet me cluster-in, krijon keyspace/tabela nëse mungojnë dhe përgatit prepared statements për insert.

Hapi 4 – Integrimi me Spark Streaming. Spark nuk shkruan direkt në Cassandra; integrimi bëhet në ForeachBatch përmes CassandraHealthRepository (CassandraCSharpDriver). Çdo batch Spark → Collect() → ProcessReadingAsync / StoreWindowBatchAsync → INSERT në Cassandra. Kjo qasje lejon kontroll të plotë të logjikës së biznesit (alarme, risk, SignalR) brenda çdo batch-i.

Hapi 5 – Integrimi me vue-client. Vue.Api lexon të dhënat historike nga Cassandra përmes REST API-së (/api/overview, /api/patients/{id}, /api/analytics, /api/alerts), ndërsa përditësimet live vijnë nga Streaming.App → SignalR → vue-client.

Skema e plotë e bazës së të dhënave (keyspace: smart_health)

Tabela patient_vitals – leximet e papërpunuara. PRIMARY KEY: ((room_number, vital_day),

recorded_at, patient_id). CLUSTERING ORDER BY (recorded_at DESC). Kolonat: patient_name, age, heart_rate, spo2, temperature, systolic_bp, diastolic_bp, respiratory_rate. Shkruhet nga Streaming.App për çdo lexim valid.

Tabela patients – regjistri i pacientëve. PRIMARY KEY: (patient_id). Përmban emrin, dhomën, moshën dhe profilin klinik (kolesterol, diabet, duhan, BMI, etj.) për modelin ML.NET. Përditësohet me UpsertPatientAsync.

Tabela alerts_log – historiku i alarmeve. PRIMARY KEY: ((room_number, alert_day), recorded_at, alert_id). Kolonat: patient_id, alert_type, severity (CRITICAL/WARNING/INFO), message, value dhe metrikat në momentin e alarmit. Shkruhet kur AlertRules gjeneron alarm.

Tabela vitals_window_agg – agregimet Spark. PRIMARY KEY: ((room_number, window_day), window_start, patient_id). Kolonat: avg_heart_rate, avg_spo2, avg_temperature, avg_systolic, avg_diastolic, avg_respiratory, sample_count. Shkruhet nga StoreWindowBatchAsync pas agregimit 5 min / slide 1 min.

Tabela ai_risk_scores – vlerësimi i rrezikut në kohë reale. PRIMARY KEY: ((patient_id, score_day), recorded_at). Kolonat: room_number, score, category, factors, plus metrikat vitale. Shkruhet nga RiskScoringEngine gjatë ProcessReadingAsync.

Tabela ml_predictions – parashikimet ML.NET. PRIMARY KEY: ((patient_id, prediction_day), recorded_at). Kolonat: room_number, risk_probability, risk_category, top_factors. Shkruhet periodikisht nga worker-i ML në Vue.Api.

Rrjedha e shkrimit Spark → Cassandra: Kafka → ReadStream → Filter → ForeachBatch → CassandraHealthRepository.InsertVitalAsync / InsertAlertAsync / InsertRiskAsync / InsertWindowAsync. Partition key-at janë zgjedhur sipas pyetjeve të dashboard-it (sipas dhomës/ditës për vitale, sipas pacientit/ditës për risk), duke optimizuar leximin e shpejtë nga vue-client.

Ndërfaqja e Vizualizimit

Ndërfaqja e vizualizimit është zhvilluar si aplikacion i pavarur vue-client (Vue 3, Composition API, Pinia, Vue Router, Vite 4, ApexCharts). Vue projekti build-ohet me npm run build dhe servoset nga Vue.Api në wwwroot. Përditësimet live merren përmes SignalR nga hub-i /healthHub (eventet: vitalsReceived, alertReceived, riskReceived, metricsReceived, mlPredictionReceived), ndërsa të dhënat historike merren nga REST API (/api/live, /api/alerts, /api/analytics, /api/system, /api/ai/predictions).

Ndërfaqja vue-client përbëhet nga shtatë faqe funksionale (pamjet e mëposhtme korrespondojnë me screenshot-et e Figures 2–8 në këtë raport):

Faqja	Funksioni
Dashboard	Treguesit kryesorë, trendet dhe alarmet e fundit
Monitorimi në kohë reale	Tabela e vitalëve me status Normal / Paralajmërim / Kritik
Dhomat	Pamje sipas dhomave 101–110
Detajet e pacientit	Grafikë historikë, statistika dhe rreziku AI (ML.NET)
Alarmet	Regjistri i alarmeve në kohë reale
Analitika	Agregimet nga dritaret 5-minutëshe të Spark
Shëndeti i sistemit	Metrikat e Kafka, Spark, Cassandra dhe API-së

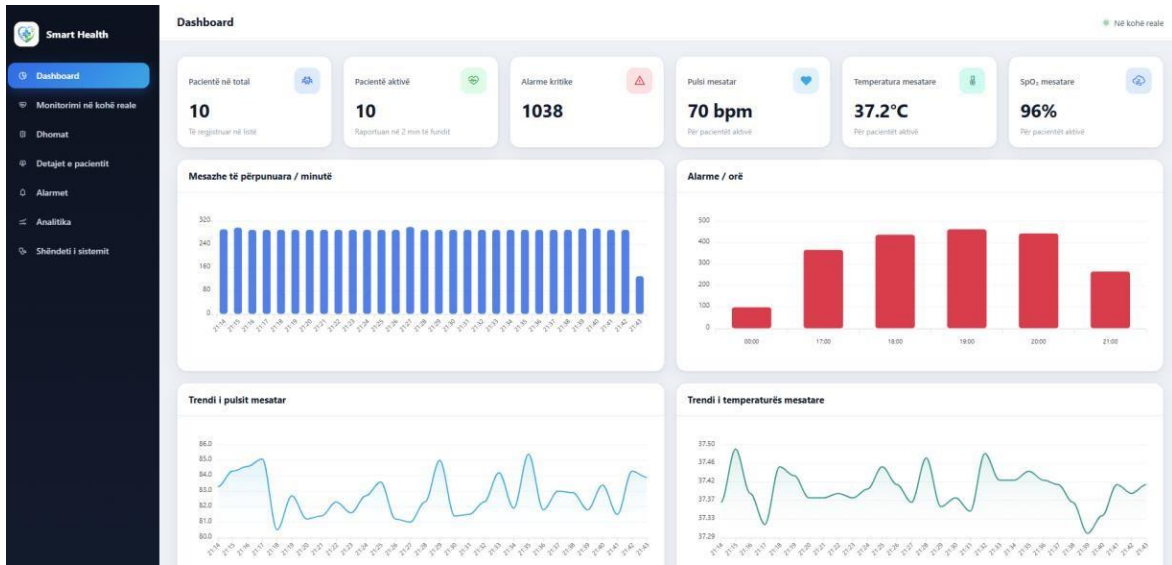


Figura 2: Dashboard kryesor me treguesit kryesorë dhe trendet

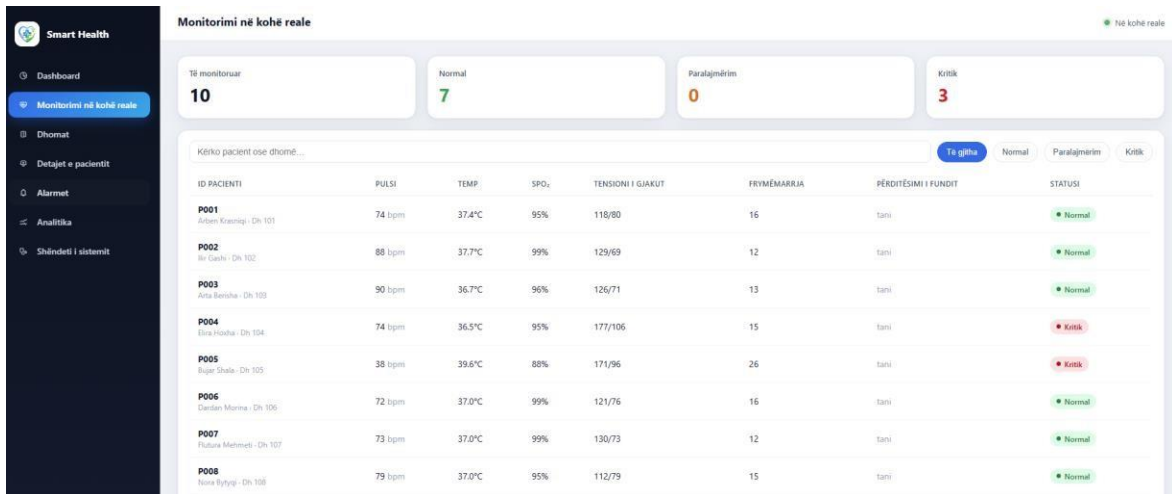


Figura 3: Monitorimi në kohë reale i parametrevë vitale

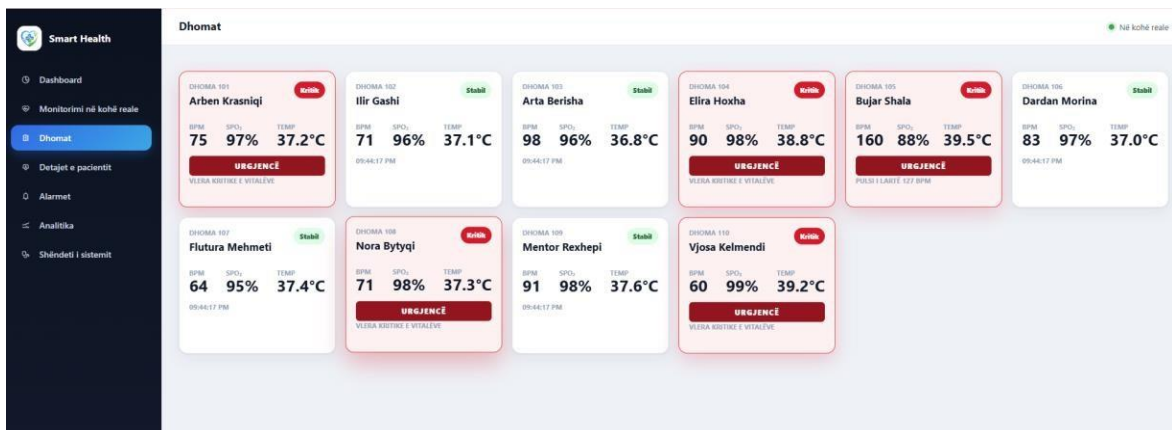


Figura 4: Monitorimi sipas dhomave 101–110

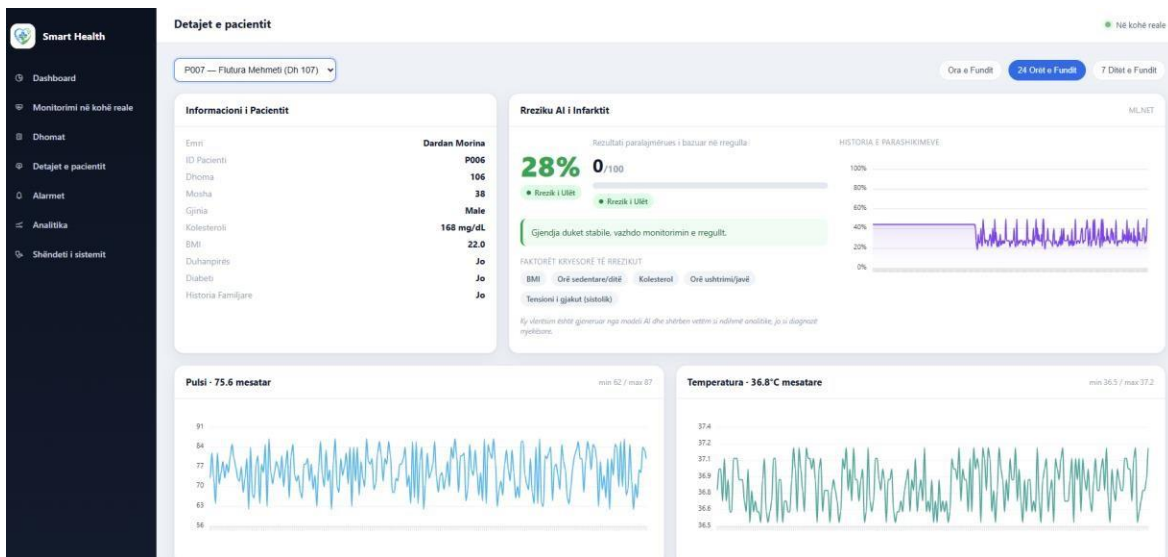


Figura 5: Detajet e pacientit me vlerësimin e rrezikut AI (ML.NET)

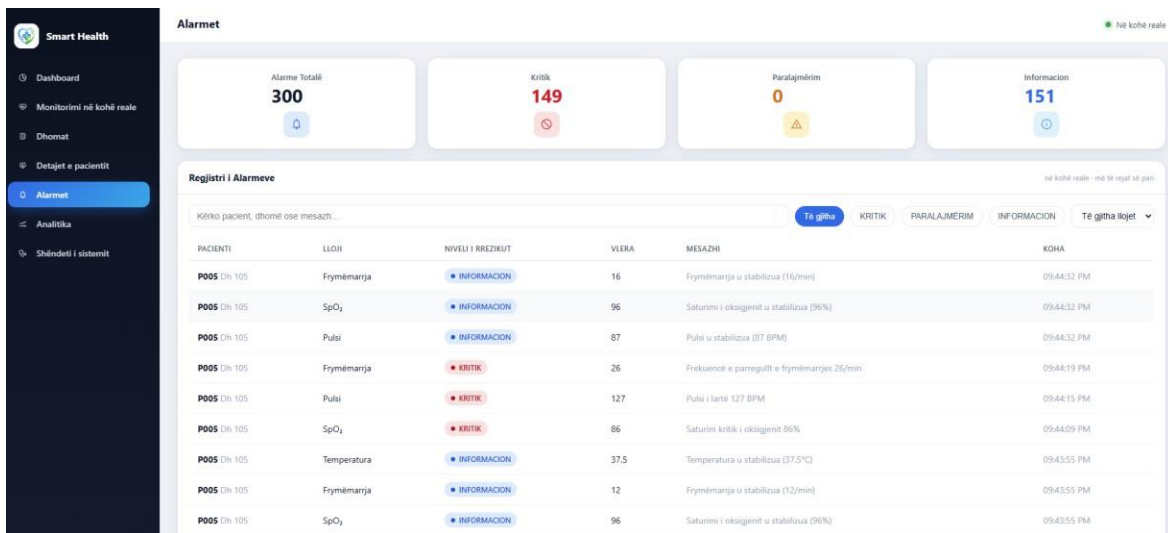


Figura 6: Qendra e alarmeve në kohë reale



Figura 7: Analitika nga dritaret 5-minutëshe të Spark

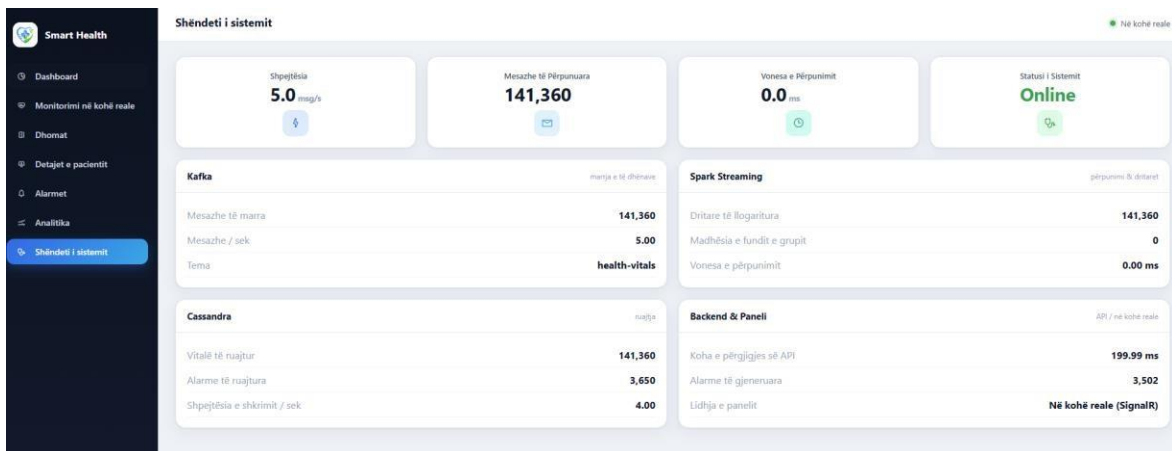


Figura 8: Shëndeti i sistemit (Kafka, Spark, Cassandra, API)

Komponentët e Avancuar

Inteligjenca Artificiale

Sistemi përfshin dy shtresa të inteligjencës artificiale. Shtresa e parë është modeli ML.NET (FastTree), i trajnuar offline nga AI.Training mbi një dataset klinik të infarktit; ky model parashikon periodikisht probabilitetin e rrezikut për secilin pacient bazuar në profilin klinik dhe leximet e fundit (ruajtur në ml_predictions). Shtresa e dytë është RiskScoringEngine, e cila vlerëson rrezikun në kohë reale gjatë përpunimit të rrjedhës dhe ruan rezultatin në ai_risk_scores.

Sistemi i Alarmeve

Motori i alarmeve AlertRules vlerëson çdo lexim vital kundrejt pragjeve klinike dhe gjeneron një alarm për secilin parametër që tejkalon normën. Nivelet e rëndësisë janë CRITICAL, WARNING dhe INFO. Rregullat mbulojnë pulsën, SpO₂, temperaturën, tensionin arterial dhe frekuencën e frymëmarrjes.

- Puls: kritik nëse < 50 ose > 120 BPM; paralajmërim nëse > 110 ose < 55 BPM.
- SpO₂: kritik nën 90%; paralajmërim nën 94%.
- Temperatura, tensioni arterial dhe frymëmarrja: pragje kritike dhe paralajmëruese të ngjashme me protokollet klinike; alarmet ruhen në alerts_log dhe shfaqen live në vue-client.

Analiza e Përformancës

MetricsCollector (në Streaming.App) mat tregues të pipeline-it në kohë reale: mesazhe të procesuara, mesazhe/sekondë, madhësia e batch-it Spark, vonesa e përpunimit (ms), dritare të agreguara dhe alarme të gjeneruara. Çdo 3 sekonda, një snapshot dërgohet te vue-client përmes SignalR (PublishMetrics) dhe shfaqet në faqen Shëndeti i sistemit.

Sfidat dhe Mësimet e Nxjerra

Gjatë ndërtimit të sistemit u hasën disa sfida teknike, trajtimi i të cilave përbën edhe mësimet kryesore të projektit.

Sfida	Si u trajtua
Garancia e moshumbjes së të dhënave nga Kafka	Checkpoint-e Spark dhe auto-offset reset (Latest) në mënyrën direct Kafka
Të dhëna jashtë rendit në agregime	Përdorimi i watermark-ut me dritare rrëshqitëse 5-minutëshe
Tepricë alarmesh	AlertRules me nivele CRITICAL / WARNING / INFO për secilin parametër vital
Komunikimi në kohë reale me ndërfaqen	Streaming.App si klient SignalR te hub-i /healthHub i Vue.Api; vue-client subscribe në eventet live

Mësimet kryesore: decoupling-u me Kafka e bën sistemin më të qëndrueshëm dhe të zgjerueshëm, ndërsa përpunimi në kohë reale kërkon balancim midis saktësisë dhe vonesës.

Përfundime dhe Rekomandime

Projekti realizon një pipeline IoT funksional nga fillimi në fund: Simulator.App → Apache Kafka → Apache Spark Structured Streaming → Apache Cassandra → vue-client (Vue.Api + SignalR). Përveç kërkesave bazë, janë integruar inteligjenca artificiale (ML.NET + RiskScoringEngine), sistemi i alarmeve (AlertRules) dhe analiza e performancës (MetricsCollector).

Bazuar në përvojën e zhvillimit, rekomandohet:

- Përdorimi i Apache Kafka si shtresa qendrore e transmetimit në sistemet IoT me volum të lartë.
- Projektimi i skemës së Cassandra-s sipas modelit të pyetjeve (query-first), për performancë optimale.
- Aktivizimi i TLS/SSL për Kafka dhe Cassandra në mjedise prodhimi.
- Shkallëzimi i Spark në cluster (YARN/Kubernetes) dhe rritja e replication factor në Cassandra.
- Integrimi i observability (Prometheus/Grafana) për metrika të pipeline-it.

Puna e Ardhshme

Sistemi mund të zgjerohet më tej me drejtimet e mëposhtme:

- Zëvendësimi i simulatorit me sensorë fizikë realë, p.sh. përmes protokollit MQTT.
- Shtimi i autentikimit dhe autorizimit me role për stafin mjekësor.
- Ritrajnimi i modelit të AI me një dataset klinik më të pasur dhe më të balancuar.
- Politika ruajtjeje afatgjatë me TTL dhe arkivim të të dhënave historike.